ANÁLISE DE VOOS NO BRASIL

Documentação técnica

1. Visão geral do projeto

O projeto Análise de Voos no Brasil foi desenvolvido para coletar, organizar e armazenar informações sobre voos e preços de passagens aéreas. A solução reúne dados provenientes de APIs externas, realiza o tratamento em Python e mantém um histórico em banco SQLite, permitindo que as informações sejam posteriormente exploradas em Python ou Power BI.

A proposta central é transformar consultas isoladas de voos e tarifas em uma base estruturada, adequada para comparação de rotas, preços, horários, escalas, duração das viagens e situação operacional dos voos.

2. Objetivos do projeto

Objetivo geral: desenvolver um fluxo automatizado de coleta e armazenamento de dados de aviação que possa servir de base para análises e visualizações.

Entre os objetivos específicos estão:

- consultar informações de status e horários de voos em aeroportos selecionados;
- obter tarifas e itinerários para rotas configuradas no sistema;
- padronizar as respostas das APIs em estruturas tabulares;
- preservar o histórico das coletas sem sobrescrever registros anteriores;
- disponibilizar os dados para análises em pandas, matplotlib e Power BI.

3. Tecnologias utilizadas

A solução foi construída com tecnologias leves e amplamente utilizadas em projetos de análise de dados.

Tecnologia	Função no projeto
Python	Executa a coleta, o tratamento e a gravação dos dados.
Requests	Realiza as requisições HTTP para as APIs externas.
Pandas	Transforma as respostas recebidas em DataFrames padronizados.
SQLite	Armazena o histórico de voos, tarifas e dados de referência.
python-dotenv	Carrega as chaves de acesso das APIs a partir de variáveis de ambiente.
Matplotlib	Permite criar gráficos complementares diretamente em Python.
Power BI	Permite construir dashboards e indicadores a partir do banco de dados.

4. Arquitetura geral da solução

A arquitetura do projeto é organizada em quatro etapas principais: obtenção dos dados, tratamento, armazenamento histórico e análise. Essa separação facilita a manutenção do código e permite que novas fontes de dados sejam adicionadas futuramente.

Fluxograma 1 - Arquitetura geral



Fonte: elaboração própria com base no repositório do projeto.

O arquivo main.py funciona como ponto de entrada da aplicação e coordena as etapas de consulta, transformação e gravação dos dados.

5. Fontes de dados e APIs

O projeto utiliza duas fontes principais, cada uma com uma finalidade diferente. Os dados coletados não representam compra ou emissão de bilhetes; são registros obtidos no momento da consulta.

Fonte	Informações utilizadas
AviationStack	Status do voo, número, companhia, origem, destino, horários previstos, duração e dia da semana.
Ignav	Preço, companhia, horários, classe, número de escalas, duração, itinerário e características da passagem.

As credenciais de acesso são mantidas fora do código-fonte e carregadas por variáveis de ambiente, reduzindo o risco de exposição de chaves no repositório.

6. Processo de coleta

A coleta é iniciada pelo main.py. O programa percorre os aeroportos e rotas configurados, consulta as APIs e grava os resultados no SQLite. Caso ocorra erro em uma rota ou aeroporto, as demais consultas podem continuar, evitando que uma falha pontual interrompa todo o processo.

Fluxograma 2 - Processo de coleta



Fonte: elaboração própria com base no repositório do projeto.

A gravação utiliza o modo de inclusão de registros, preservando coletas anteriores. Esse comportamento é essencial para análises históricas, pois permite comparar dados obtidos em momentos diferentes.

7. Tratamento e padronização dos dados

As respostas das APIs possuem estruturas diferentes. Antes do armazenamento, o projeto seleciona os campos relevantes e os converte para um formato padronizado em DataFrames.

- status de voo é convertido para rótulos em português;
- horários são utilizados para calcular duração e identificar o período do dia;
- preços são mantidos em formato numérico para cálculos e também em formato textual para exibição;
- a quantidade de escalas é calculada a partir dos segmentos do itinerário;
- é calculada a antecedência entre a data da coleta e a data da viagem;
- valores ausentes ou inválidos são tratados para evitar a interrupção de toda a coleta.

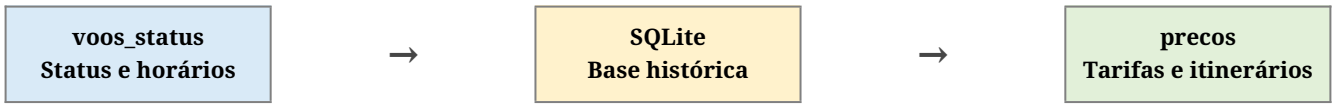
Essa etapa reduz diferenças entre as fontes e deixa os dados prontos para consultas, filtros e construção de indicadores.

8. Banco de dados e organização das informações

O projeto utiliza o arquivo voos_brasil.db, criado em SQLite. A estrutura é definida pelo arquivo db/schema.sql e separa dados de referência, histórico operacional e histórico de preços.

Tabela / View	Conteúdo
aeroportos	Códigos IATA, nomes, cidades, estados, coordenadas e fuso horário.
companhias	Cadastro de companhias aéreas utilizado como referência.
voos_status	Histórico dos voos consultados na AviationStack.
precos	Histórico de tarifas e itinerários retornados pela Ignav.
resumo_rotas	Estrutura preparada para indicadores agregados por rota e período.
vw_passagens_atuais	View com a coleta mais recente de tarifas por origem, destino e data de viagem.

Fluxograma 3 - Organização dos dados



Fonte: elaboração própria com base no repositório do projeto.

A view vw_passagens_atuais simplifica o consumo dos dados no Power BI, pois seleciona a coleta mais recente de cada combinação de origem, destino e data de viagem e organiza as opções por preço.

9. Estrutura principal do projeto

Os arquivos foram separados por responsabilidade, o que facilita identificar onde cada parte do fluxo é executada.

Arquivo / pasta	Responsabilidade
main.py	Coordena as consultas e grava os resultados no banco SQLite.
extract/aviationstack.py	Consulta e transforma informações de status dos voos.
extract/ignav.py	Consulta e transforma preços e itinerários.
db/init_db.py	Cria ou atualiza o banco e cadastra os aeroportos iniciais.
db/schema.sql	Define tabelas, índices e a view utilizada nas análises.
common/common/aeroportos.py	Mantém catálogo auxiliar e cálculo de distância entre aeroportos.
requirements.txt	Lista as bibliotecas necessárias para executar o projeto.

10. Configuração e execução

Para executar o sistema, é necessário preparar o ambiente Python, configurar as credenciais das APIs e inicializar o banco de dados.

1. Instalar as dependências com: python -m pip install -r requirements.txt
2. Configurar as variáveis AVIATIONSTACK_API_KEY e IGNAV_API_KEY no ambiente do projeto.
3. Criar ou atualizar o banco com: python db/init_db.py
4. Revisar no main.py os aeroportos, rotas e a data que serão consultados.
5. Executar a coleta com: python main.py

Existe também uma opção de recriação completa do banco. Como essa ação remove o histórico anterior, ela deve ser utilizada somente quando a intenção for iniciar uma base do zero.

11. Integração com Power BI e visualização

Após a coleta, o banco SQLite pode ser utilizado como fonte para relatórios. No Power BI, os dados podem ser carregados por meio de um script Python que consulta as tabelas e a view do projeto.

A view vw_passagens_atuais é a principal estrutura para visualizações de preço, enquanto voos_status permite trabalhar com informações operacionais dos voos. A tabela aeroportos pode complementar os relatórios com nomes, localidades e coordenadas.

Com essa organização, uma nova coleta pode ser refletida no dashboard apenas atualizando a fonte de dados, sem necessidade de reconstruir os visuais.

12. Perguntas orientadoras da análise

Com base nos gráficos produzidos a partir dos registros coletados, a análise busca responder a três perguntas principais:

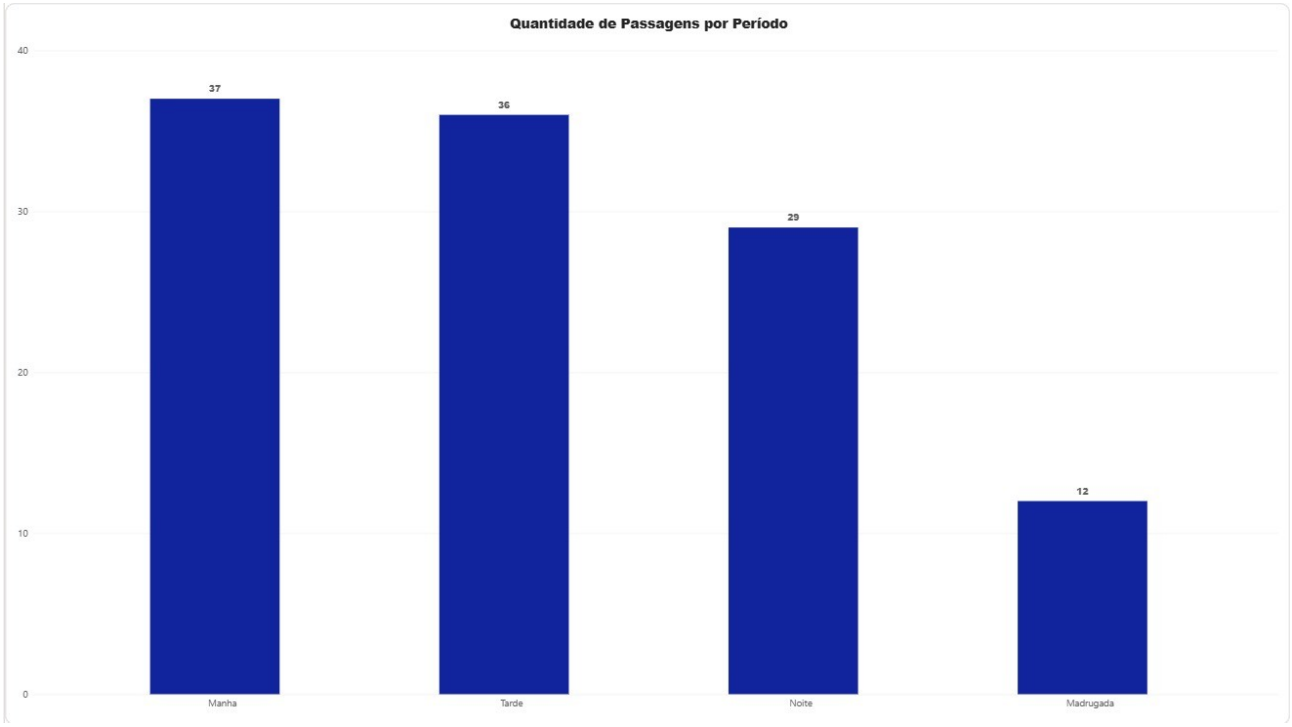
- Em qual período do dia foram encontradas mais opções de passagem?
- Qual companhia aérea apresentou o menor preço médio das passagens na amostra?
- Qual companhia aérea aparece com maior frequência entre as passagens analisadas?

Os gráficos apresentados a seguir sintetizam os resultados dessas três comparações para o conjunto de dados visualizado.

13. Resultado esperado

Os gráficos abaixo mostram os resultados obtidos no painel apresentado, com base em 114 opções de passagem. As visualizações permitem comparar períodos do dia, preços médios e frequência das companhias na amostra.

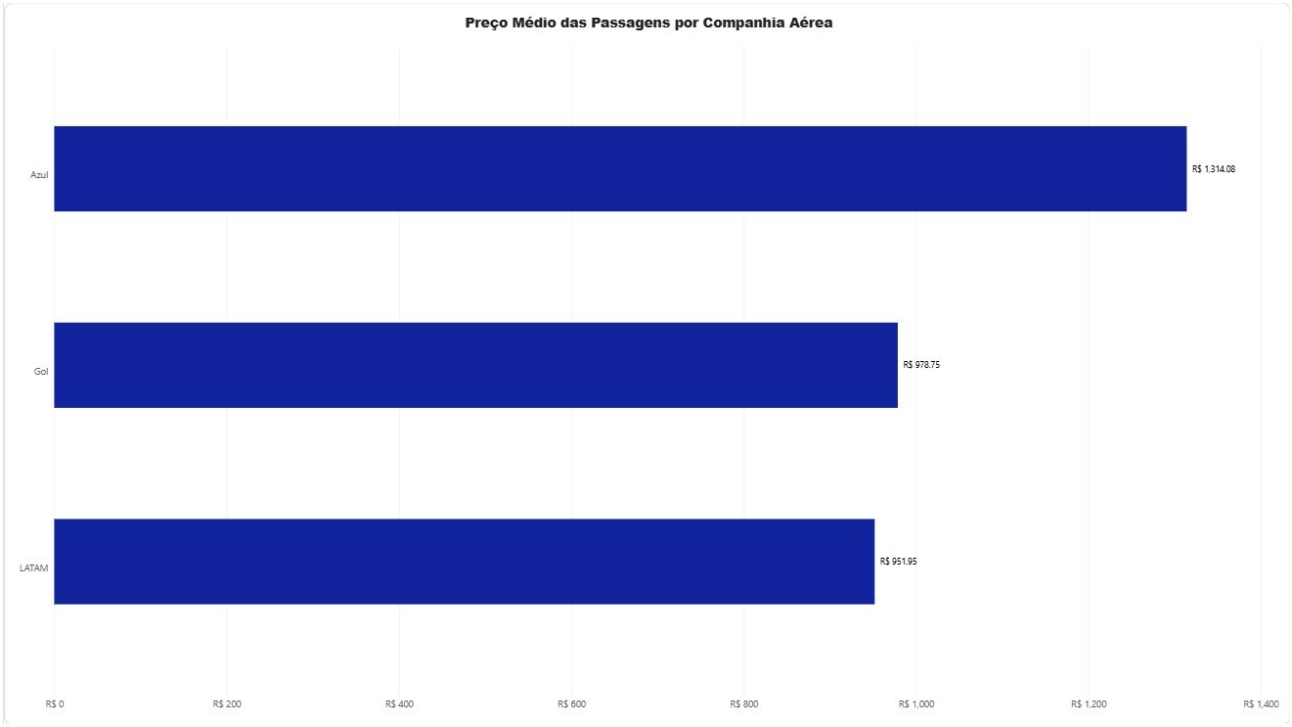
Figura 1 - Quantidade de passagens por período



Fonte: gráficos fornecidos pelos autores do projeto.

O período da manhã apresenta o maior número de opções (37), seguido pela tarde (36), noite (29) e madrugada (12).

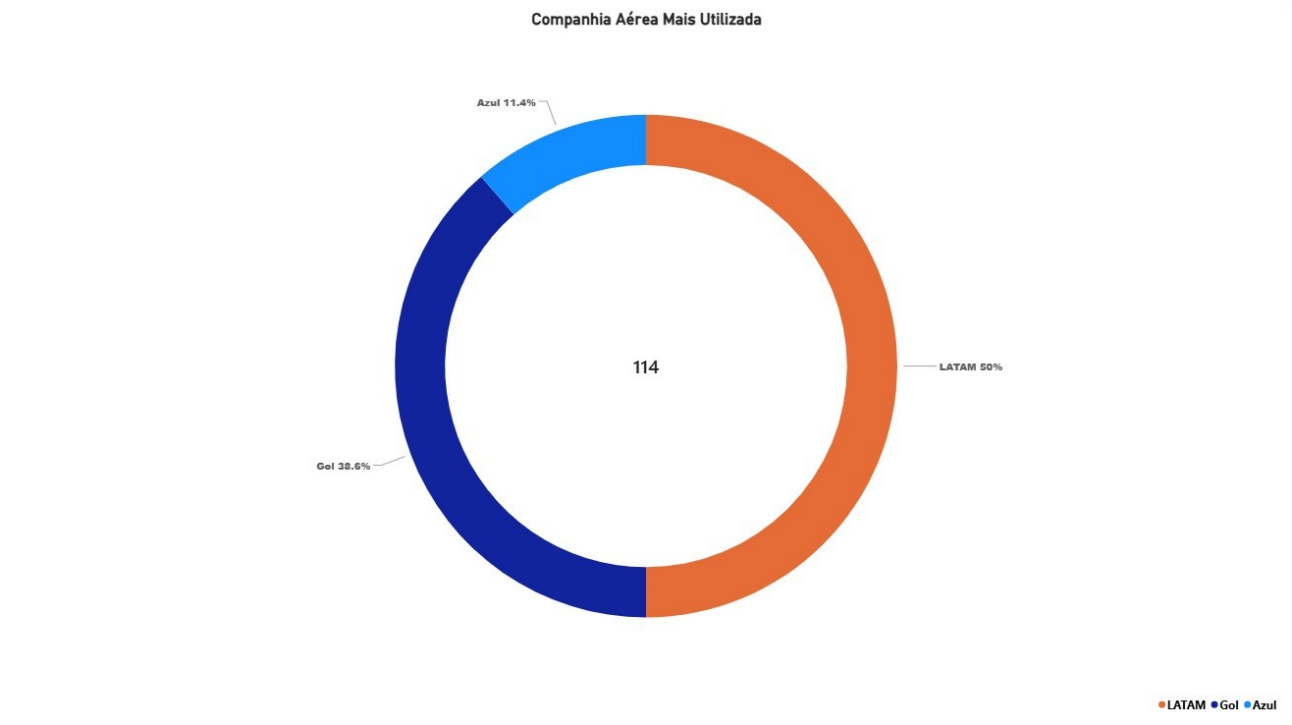
Figura 2 - Preço médio das passagens por companhia aérea



Fonte: gráficos fornecidos pelos autores do projeto.

Na amostra apresentada, a LATAM possui o menor preço médio (R$ 951,95), seguida pela Gol (R$ 978,75) e pela Azul (R$ 1.314,08).

Figura 3 - Companhia aérea mais frequente na amostra



Fonte: gráficos fornecidos pelos autores do projeto.

Entre os 114 registros apresentados, a LATAM corresponde a 50,0%, a Gol a 38,6% e a Azul a 11,4%. Esses percentuais se referem às opções coletadas, não à participação no mercado.

14. Conclusão

O projeto Análise de Voos no Brasil apresenta um fluxo completo de dados, desde a consulta a serviços externos até a disponibilização das informações para análise. Python e pandas são responsáveis pela integração e transformação; o SQLite mantém o histórico das coletas; e ferramentas como Power BI e matplotlib permitem transformar os registros em indicadores e visualizações.

A principal contribuição do projeto é a criação de uma base histórica estruturada. Com novas coletas ao longo do tempo, torna-se possível estudar o comportamento das tarifas e das rotas de maneira mais consistente, além de servir como ponto de partida para automações e análises mais avançadas.

Referências

- Repositório do projeto: https://github.com/YanAugusto2031/Analise-de-voos
- AviationStack - documentação: https://aviationstack.com/documentation
- Ignav - documentação: https://ignav.com/docs/one-way
- Python - documentação oficial: https://docs.python.org/
- pandas - documentação oficial: https://pandas.pydata.org/docs/
- SQLite - documentação oficial: https://www.sqlite.org/docs.html